PART 1 OF 2 ARCHITECTURE REVIEW DRIVE LAUNCHER

Drive Launcher As-Built

How the app works today, traced through the code: the hosting session, the lock, the file watcher, the manifest and the file transfers. Problems are marked in red on the diagrams and listed at the end with their file and line.

DRAWING	STATUS	REVISION
Session, sync and UI	As built: what the code does now	rewritten-manifest @ 6d97a0a + uncommitted changes in 3 files
DATE	**SCOPE**	**COMPANION**
14 October 2026	electron/ (main.ts + 7 services), src/ (24 components, 4 stores)	Part 2 · Drive Launcher Blueprint

01 The short version

The idea holds up. Google Drive as the one shared copy of the world, a lock file that says who is hosting, and a single button that downloads, starts, stops and uploads is a sound design for a group of friends who share a server. The screens are sensible too. What's fragile is the structure underneath: no single piece of code owns "the session", the app's state is spread over 18 module-level variables, and the new manifest code is wired in halfway.

Five things matter most:

- **The new transfer code can't move any files yet.** The list of files to transfer always comes out empty F1, manifest.json is never uploaded F2, per-file versions never change F3, and results are saved under the wrong keys F4. A session that ends with an upload reports success while nothing reaches Drive.
- **The lock stops protecting the world when it matters most.** Its lease runs out during long downloads and uploads F6, and Start only refuses when the status is "online" F7. A second member can start hosting while the first is still uploading.
- **Stopping doesn't wait for Minecraft.** The upload begins while the server is still saving F5, and the stop sequence runs twice F9.
- **Nothing owns the session.** main.ts, serverService, childrenProcesses and the renderer each keep their own answer to "is a server running?" (section 10).
- **The build is broken right now.** A rename was finished in the services but not in main.ts or the Manifest modal, and chokidar is missing from package.json F20.

8 CRITICAL 12 HIGH 15 MEDIUM 6 LOW

> HOW TO READ THIS
>
> Sections follow the life of a hosting session. A red tag like F6 points to a finding in the appendix. References like `serverService.ts:24` point to a line in your current working tree, uncommitted changes included. Part 2 shows the structure that fixes these at the root.

02 What already works well

Start with what to keep. These are good decisions, and several of them are ones experienced developers often get wrong.

- **One button for the whole session.** Hiding download → launch → stop → upload behind Power/Stop is the right product call. The rest of this review is about making that button trustworthy.
- **A lock file with an expiry and a heartbeat** `serverService.ts:150`. Drive has no real locking, and a lease that has to be renewed is the right basic shape for "who is hosting". It needs an owner ID and continuous renewal, not a different idea.
- **Atomic manifest writes.** Writing manifest.tmp and renaming it over manifest.json `watcher-manifest.ts:155` means a crash can never leave a half-written manifest.
- **Detecting offline changes** by comparing a fresh scan with the last manifest `watcher-manifest.ts:105`. This is the core of a correct sync design, and Part 2 builds on it.
- **Paging through folder listings** with `nextPageToken` `fileTransferService.ts:79` instead of assuming one page is everything.
- `Promise.allSettled` **for deletions** `fileTransferService.ts:98`, so one failed delete doesn't hide the others.
- **Shared types** in src/lib/types.ts used by both processes, a promise-based confirm hook (`useConfirm`), progress messages with major and minor importance, and a careful tray and quit flow.

03 The big picture

Electron runs your code in two processes. The **renderer** is a Chromium page running React. The **main process** runs Node.js and is the only part that can touch the disk, start Java or hold the Google token. They talk over IPC: the renderer invokes a named channel and awaits the answer, and the main process can push events back.

Here, preload.ts hands the renderer `ipcRenderer` itself, so components call `window.ipcRenderer.invoke('start-server', …)` with plain strings. There are 43 request channels and 12 push events, and every one of them is registered in main.ts, which also creates the window and tray, runs the quit flow, orchestrates hosting, runs the heartbeat and parses the player list.

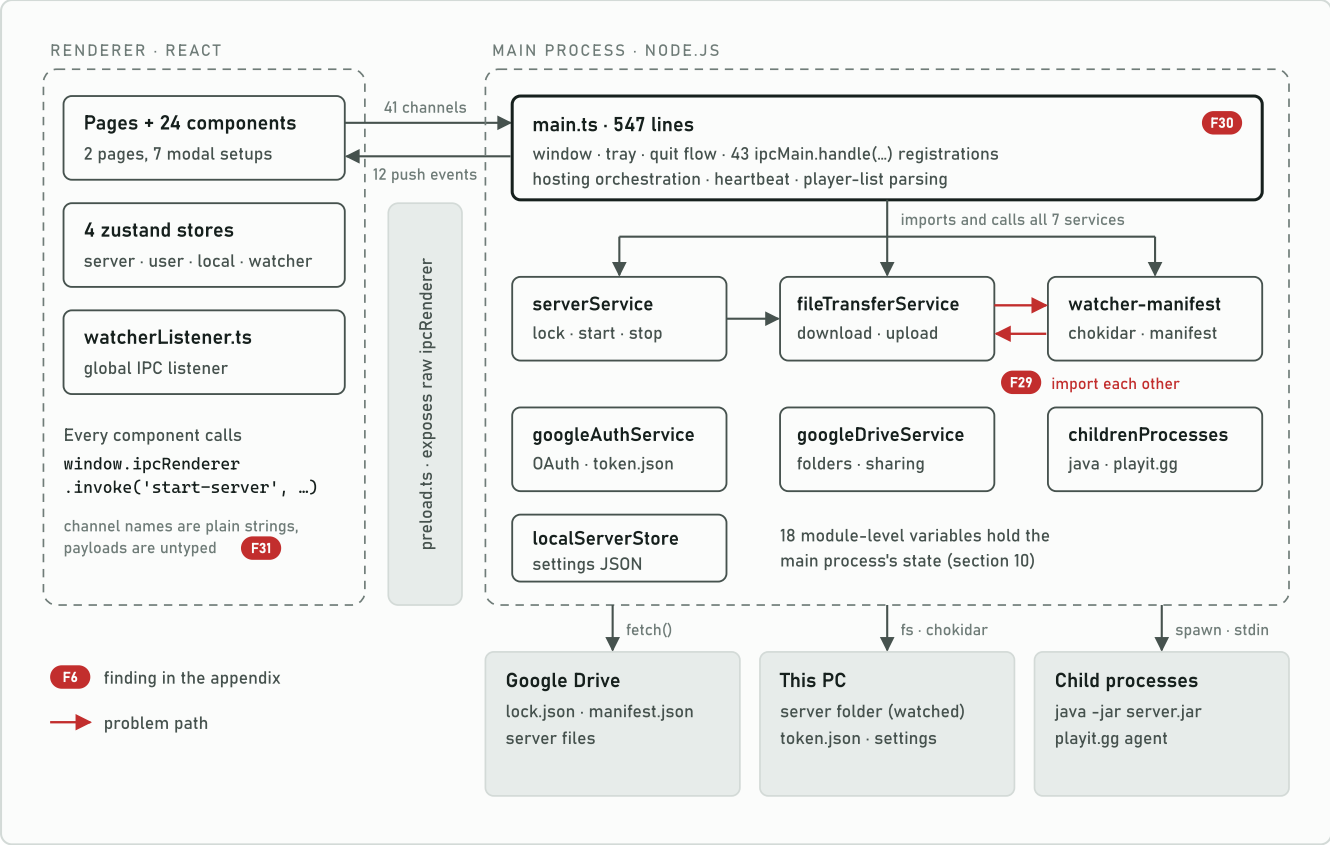


Figure 1. Process and module layout today. main.ts is the hub of every feature, the seven services call each other freely, and two of them import each other F29. The renderer reaches the main process through string-named channels F31.

The 43 request channels, by purpose

AREA	COUNT	CHANNELS
Sign-in and scopes	6	`google-login` `google-get-user` `google-is-logged-in` `google-logout` `is-request-allowed` `request-drive-scope`
Servers and sharing	8	`drive-create-server` `drive-delete-server` `drive-get-root` `rename-server` `drive-invite-user` `drive-remove-permission` `join-by-id` `get-joined-folders`
Hosting session	8	`start-server` `stop-server` `send-server-command` `is-server-running` `get-current-players` `get-max-players` `get-server-lock` `confirm-quit-response`
File transfer	4	`sync-server` `upload-server-folder` `get-files-to-download` `get-files-to-upload` (the last two are marked temporary)
Watcher and manifest	10	`start-watcher` `stop-watcher` `is-watcher-running` `get-watcher-updates` `get-tracked-files` `mutate-tracked-files` `get-blacklist` `set-blacklist` `write-manifest` `open-manifest`
Settings and OS	7	`get-local-variable` `set-local-variable` `get-server-path` `set-server-path` `choose-file-directory` `open-folder` `open-external-link`

Three of these connections are dead: `get-server-path` and `open-external-link` are handled but never called, and the `playitgg-link` event is sent but nothing listens to it (the address is read from lock.json instead). `write-manifest` is still called by the Manifest modal but points at a function that no longer exists F20.

04 The UI, screen by screen

The window is a fixed 800 × 600, split into three columns: a 15% members panel, a 70% main area and a 15% menu. Minimizing or closing sends the app to the tray. Quitting asks for confirmation and, if you're hosting, stops the server and uploads before exiting.

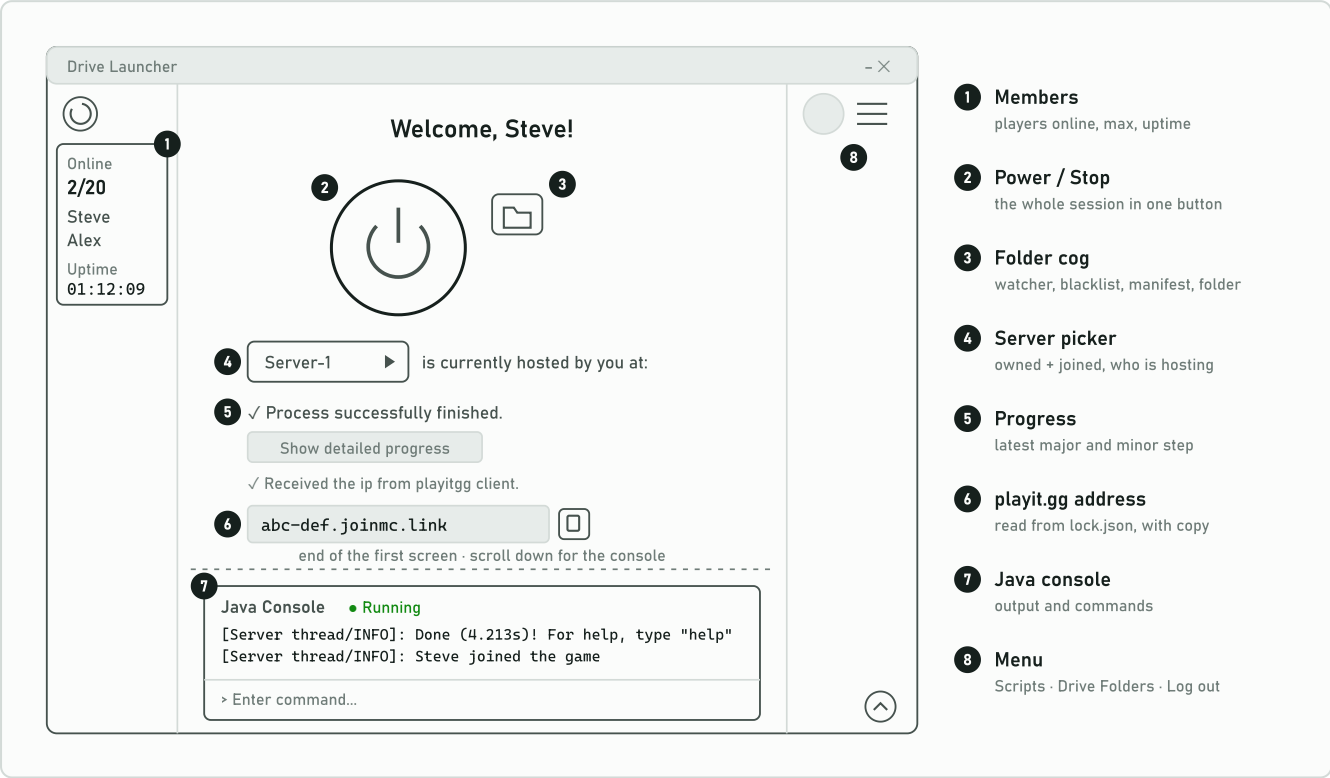


Figure 2. The main window. The Java console sits one screen below the Power button; a scroll-to-top button appears once you scroll down.

```
App  (HashRouter: "/" and "/main")
├─ LoginPage ............................... google-login, google-is-logged-in
└─ MainPage ................................ user, scopes, settings, servers; quit dialog
   ├─ MembersSection ....................... get-current-players, get-max-players
   ├─ GreetingSection ...................... show-progress, hide-progress
   │  ├─ AppCockpit ........................ start-server, stop-server, drive-create-server
   │  │  └─ DirectoryMenu (folder cog)
   │  │     ├─ Modal: File Watcher | Blacklist ... start-/stop-watcher, get-/set-blacklist
   │  │     ├─ Modal: Upsert | Remove ............ get-watcher-updates, write-manifest
   │  │     └─ Directory ....................... open-folder
   │  ├─ SelectServer ...................... get-server-lock, set-local-variable
   │  ├─ StartupProgress ................... startup-progress
   │  │  └─ Modal: ProgressModal ............ get-files-to-download / -upload (test)
   │  ├─ PlayitggLink ...................... reads lockStatus.publicIp
   │  └─ ServerConsole ..................... server-output, send-server-command
   └─ BurgerMenu ........................... google-logout
      ├─ Modal: Script Options .............. get-/set-local-variable, choose-file-directory
      └─ Modal: Your Folders | Joined Folders
         ├─ ServerSlot (per server) ......... rename-server, sync-server, upload-server-folder,
         │                                    drive-delete-server, set-server-path
         │  └─ PermissionsList ................ drive-remove-permission
         │     └─ Modal: InvitationInterface .. drive-invite-user
         └─ JoinServer ..................... join-by-id
```

Figure 3. Component tree with the channels each part uses. Modals open from three places: the folder cog, the burger menu and the progress panel.

What the sync looks like to a user today

The sync is visible in only two places, and both are developer tools rather than user features. The folder cog's File Watcher and Manifest tabs show internal lists (Upsert, Remove) and need manual Start and Write buttons. The Download and Upload buttons in Drive Folders run a full transfer with no lock check F10. Neither answers the questions a player has before pressing Power: *what changed, what will be transferred, and is it safe to start?* Part 2 proposes three panels for exactly that.

Errors surface as `alert()` boxes. When a step fails halfway, spinners can keep spinning: a failed or hung playit.gg wait leaves the Power button loading F21, and a failed start leaves the progress panel open F38.

---

05 A hosting session, step by step

This is the path that matters most: everything between pressing Power and the world being safely back on Drive.

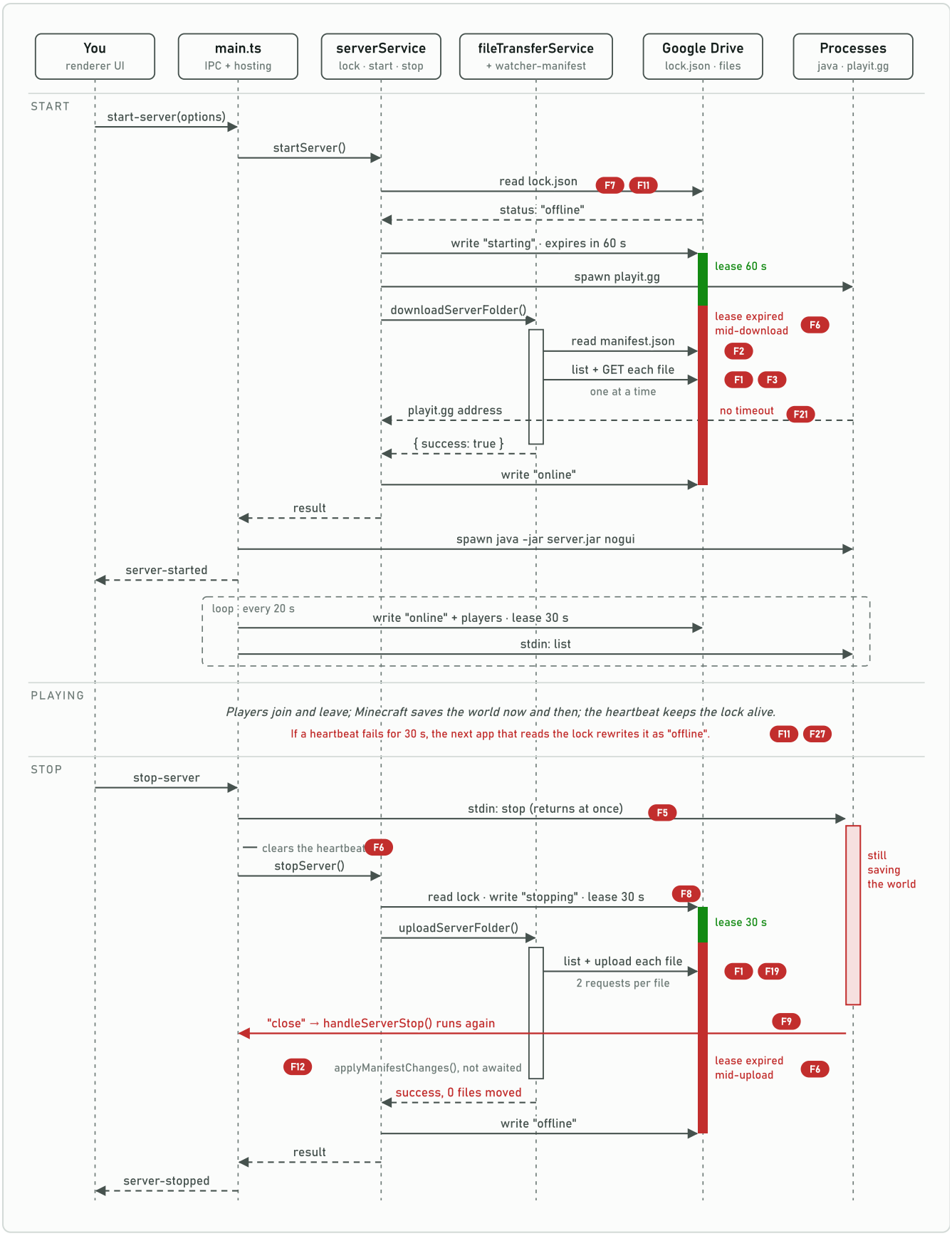


Figure 4. One complete session as the code runs it today. Time flows downward and dashed arrows are replies. The green and red bars on the Google Drive lifeline show when the lock protects the world and when it no longer does.

Starting

`startServer()` reads lock.json and refuses only when the status is "online" F7. It then writes "starting" with a 60-second expiry and launches playit.gg. The download and the wait for the playit.gg address run side by side in a `Promise.all`. The wait has no timeout, and if it fails the error escapes without resetting the button or the lock F21. When both are done the lock becomes "online" and main.ts spawns `java -jar … nogui`.

The read at the start can also write: if lock.json is missing or expired, `getServerLock()` rewrites it as "offline" with the reader's name as the host F11.

Playing

Every 20 seconds the heartbeat rewrites lock.json as "online" with a fresh 30-second expiry and the player list, and sends `list` to the server so the next heartbeat has fresh names. The heartbeat is the only thing keeping the lock alive. If it fails for 30 seconds (a Wi-Fi drop, a laptop going to sleep), the next member whose app reads the lock marks it "offline" F11, and the failure itself becomes an unhandled promise rejection that nobody sees F27.

Stopping

`killServer()` writes `stop` to Java's input and returns at once. Nothing waits for Java to exit, so the upload starts while Minecraft is still saving chunks F5. The heartbeat is cleared before the upload F6, so the 30-second "stopping" lease is never renewed. If the lock lapsed at any point during the session, `stopServer()` answers "This server is not being hosted" and skips the upload entirely F8. When Java finally exits, its `close` handler calls `handleServerStop()` a second time F9; that run clears the hosting flag and tells the UI the server has stopped while the first upload may still be running.

---

06 The lock

lock.json is written by `updateLockFile()` and looks like this:

```
{
  "hostName": "Steve",
  "hostEmail": "steve@example.com",
  "startedAt": "2026-10-14T17:02:11.000Z",
  "expiresAt": "2026-10-14T17:31:40.000Z",
  "publicIp": "abc-def.joinmc.link",
  "status": "online",
  "onlinePlayers": ["Steve", "Alex"]
}
```

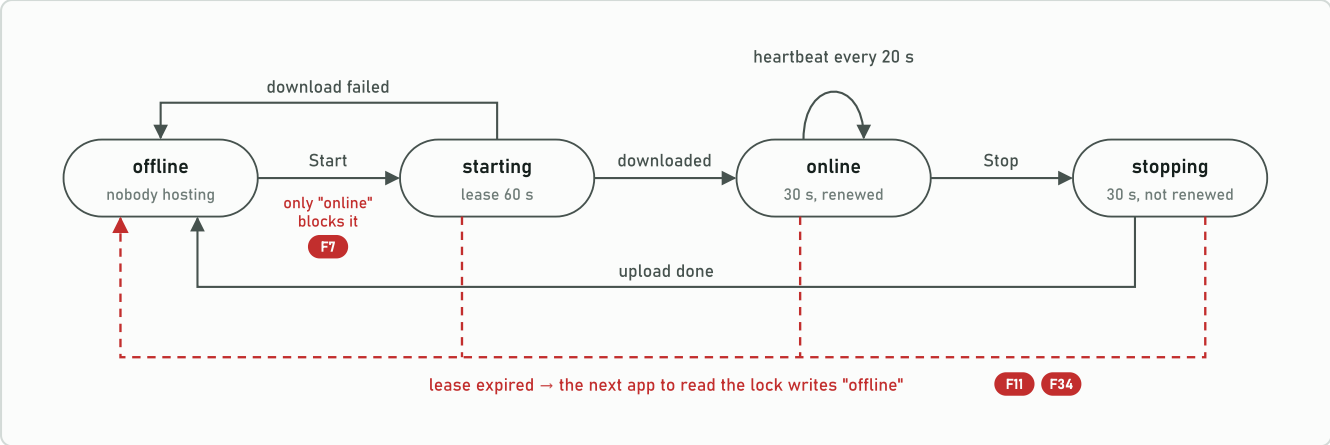


Figure 5. The lock's life cycle as implemented. Black arrows are the intended path. The red dashed path is the one nobody designed: whenever a lease runs out, the next app that reads the lock rewrites it as "offline", whatever was happening.

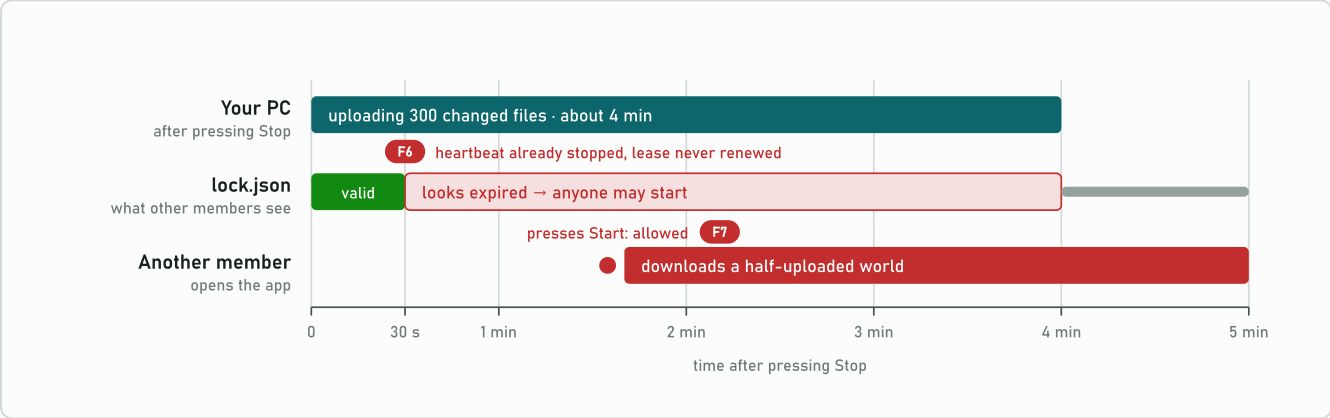


Figure 6. Example timing after pressing Stop with 300 changed files to upload. The 30-second "stopping" lease is written once and never renewed, because the heartbeat was stopped first. The same gap exists at the start: a download longer than 60 seconds outlives the "starting" lease.

What's structurally wrong with it

- **Reading writes.** `getServerLock()` creates the lock when it's missing and overwrites it when it has expired, using the reader's own name as `hostName` F11. Every member's app, including ones that are only looking, can write the lock. A function named get… should only read.
- **The lease is shorter than the work it protects**, and the renewal stops right before the longest step F6. A lease has to be renewed for as long as you rely on it.
- **Ownership can't be checked.** There's no session ID, so the app can't tell "my lock" from "a lock with my email left over from a crash". Nothing re-reads the lock after writing it, so two people who press Start in the same second can both win. And expiry is judged by each PC's own clock F34.

07 The file watcher

chokidar watches the server folder. The first scan reports every existing file as an `add` event (`ignoreInitial: false`). When the scan finishes it emits `ready`, and from then on each change is

reported once the file has been quiet for 3 seconds (`awaitWriteFinish`). Every event updates three collections and sends one IPC message to the renderer.

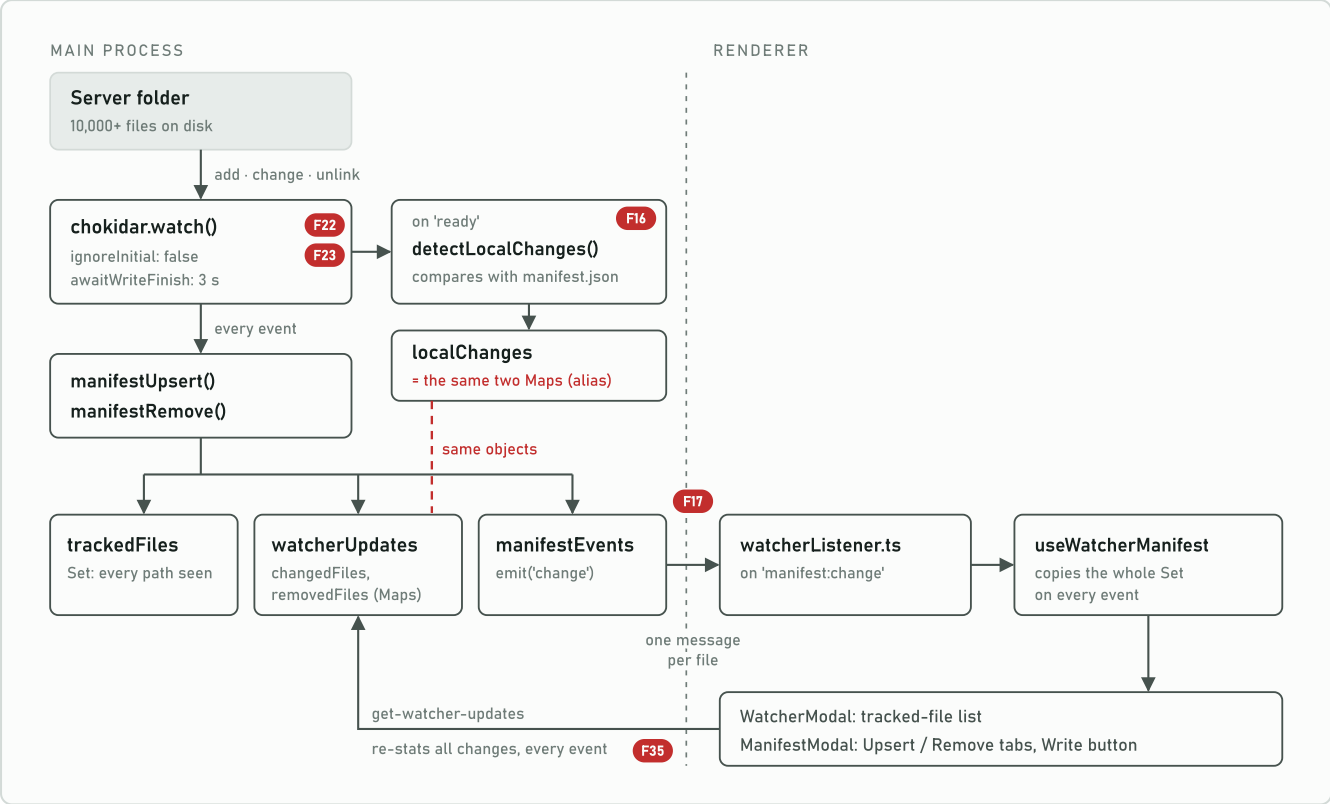


Figure 7. Watcher data flow. One file event fans out into three main-process collections and one IPC message, and the renderer keeps its own copy of the list.

The good idea

`detectLocalChanges()` runs after the first scan. It walks the manifest and compares each entry's modified time with what's on disk: the same time means unchanged, a different time means the file changed while the app was closed, and an entry with no file on disk means it was deleted. Comparing a fresh scan against a saved snapshot is how real sync tools find offline changes, and it's the foundation of Part 2.

What gets in the way

- **The watcher only runs when started by hand** from the File Watcher modal; the hosting session never starts it. At the end of a normal session `watcherUpdates` is usually empty.
- **The first scan floods the app** F17. 10,000 files produce 10,000 IPC messages; the renderer copies its whole Set for each one (about 50 million element copies for 10,000 files), and an open Manifest modal re-requests the list, re-statting every changed file, after every event.
- **Downloaded files will look changed** F16. A download gives the file a new modified time while the manifest keeps the uploader's time, so the next comparison flags every downloaded file. If the watcher runs during a download, its events arrive 3 seconds late, after the list was cleared.
- **The ignore rules are fragile.** manifest.json and lock.json aren't ignored on a fresh install F22, nested files picked with Browse never match F23, and the list is shared by every server yet private to each PC F24.

- **Three collections with overlapping meanings.** `trackedFiles` is everything seen, `watcherUpdates` is changes since the last write, and `localChanges` isn't a third collection at all: it holds the same two Map objects as `watcherUpdates`. Overlapping names like these are a sign the model isn't settled yet.

---

08 The manifest

manifest.json is meant to describe the synced state of the folder: one global version number plus an entry for each file. In the current code, a file gets an entry only after it has been transferred successfully. If F1 were fixed, an upload would record this:

```
{
    "version": 4,
    "updatedAt": 1760461500000,
    "authorName": "Steve",
    "authorEmail": "steve@example.com",
    "files": {
        "level.dat":  { "version": 0, "size": 4410,    "modifiedAt": 1760461180000 },
        "r.0.0.mca":  { "version": 0, "size": 6291456, "modifiedAt": 1760461172000 },
        "r.-1.0.mca": { "version": 0, "size": 5242880, "modifiedAt": 1760461170000 }
    }
}
```

TWO PROBLEMS IN THREE LINES

The keys are bare file names F4. A modern world has an `r.0.0.mca` in world/region, world/entities and world/poi, plus more in the Nether and End folders, and they would all share one entry. And every per-file version is 0 F3: `manifestUpsert()` writes 0, and the step that used to increase it (`writeLocalManifest`) was removed in the refactor.

How the two manifests are compared

SITUATION	WHEN DOWNLOADING	WHEN UPLOADING
No manifest.json on Drive	Error: "could not be found"	Upload every file in the local manifest
Local version is higher	Warn: Drive may be out of date	No check
Drive version is higher	No check	Warn: your copy may be out of date
Otherwise	Files with a higher per-file version, or missing locally	Watcher changes, plus files missing on Drive

Because manifest.json is never uploaded F2, the first row is the one that runs today: every download fails, and every upload tries to send everything.

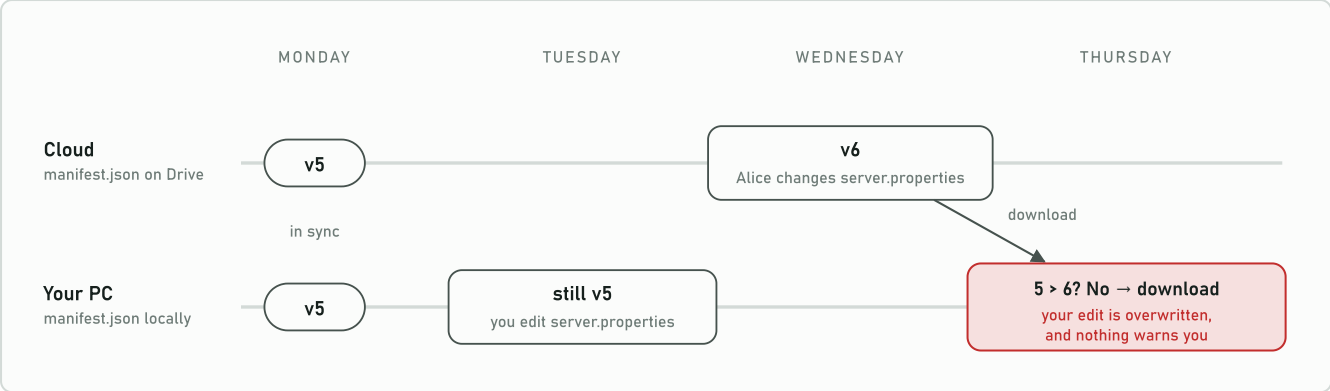


Figure 8. One number per side can't describe two people changing the world from the same starting point. Uploads are protected by the version check and downloads aren't. Changes made while the app was closed are found by `detectLocalChanges()` but never consulted F15.

A single counter can say which side is newer only while history is a straight line. As soon as two copies change from the same starting point, you need that starting point, the **base**, to tell "I changed this" from "they changed this". Part 2 keeps three snapshots (base, local and remote) and compares them file by file using content hashes instead of counters.

09 The transfer path

Both directions share one shape: decide what to move, turn the list of paths into a folder tree, walk the tree on Drive one folder at a time, move each file, record the result and save the manifest.

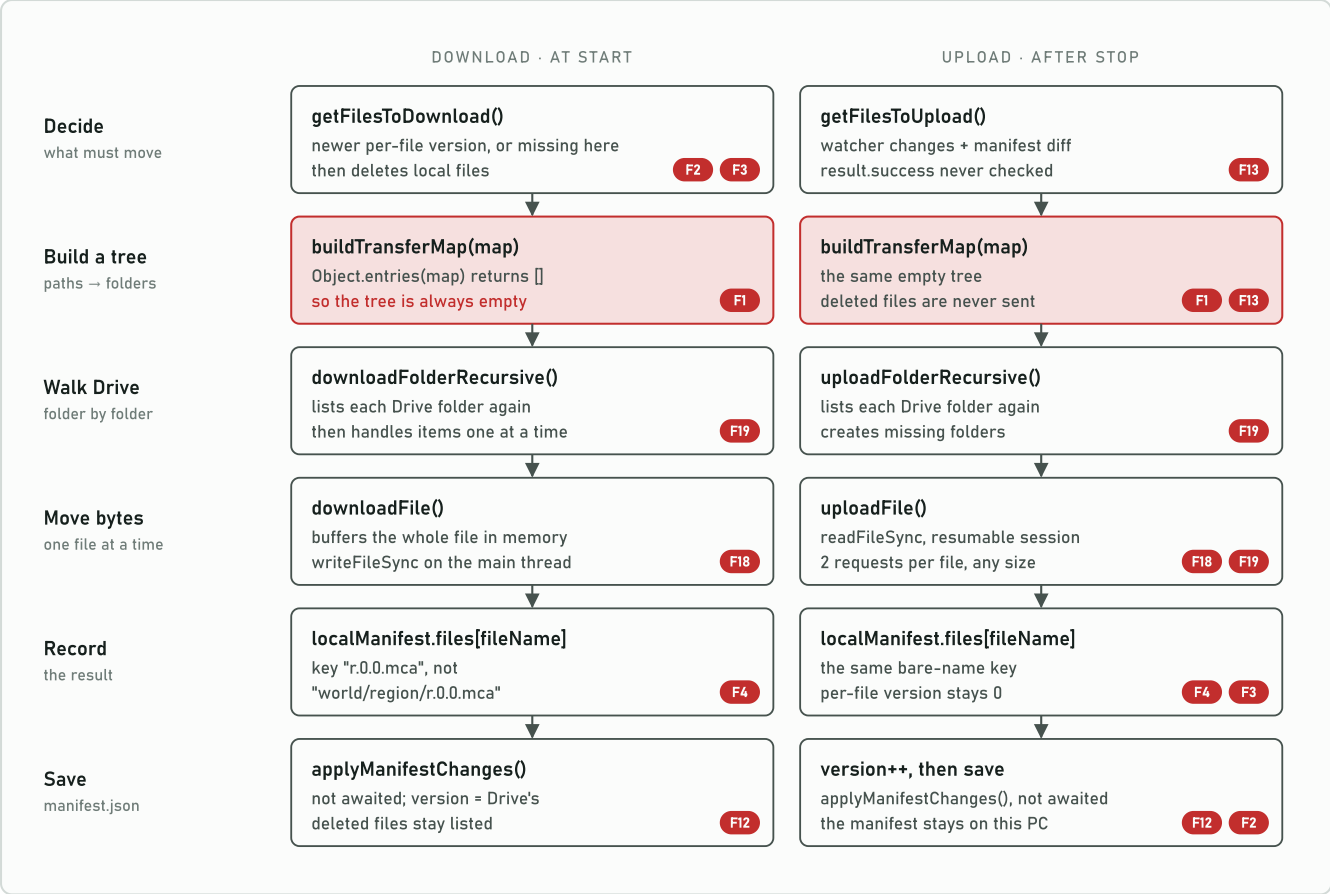


Figure 9. The download and upload pipelines side by side. Each stage makes sense on its own; together, the first two guarantee that nothing moves F1.

What happens if you host today

Tracing one real run by hand is the fastest way to check a refactor. Here is the current working tree with default settings, setting the three compile errors aside.

STEP	WHAT THE CODE DOES	RESULT
Case A · Download on (the default)		
1	Power → `startServer()`	Lock read, "starting" written, playit.gg launched
2	`downloadServerFolder()` → `getFilesToDownload()`	`readDriveManifest()` finds no manifest.json on Drive and returns an error
3	`startServer()` handles the failure	Lock set back to "offline", playit.gg killed, error shown in an alert; the progress panel stays open F38
=	**Outcome**	**You can't host while Download is checked**, unless someone placed a manifest.json on Drive by hand.
Case B · Download off, Upload on		
1	Session runs; you press Stop	The upload begins while Java is still saving F5
2	`getFilesToUpload()`	No Drive manifest, so "upload every file in the local manifest"
3	`buildTransferMap(map)`	`Object.entries(map)` is `[]`, so the tree is empty F1
4	`uploadFolderRecursive()`	Lists the server folder once, finds nothing to do, reports no failures
5	Afterwards	Version increased, manifest.json saved locally, lock set to "offline", UI says the server stopped
=	**Outcome**	**The session's changes never reach Drive, and nothing tells you.** The next host plays an older world.

What a typical upload would cost

Suppose a session changed 300 files spread over 40 folders (region, entities, poi, playerdata, stats, advancements and so on). With F1 fixed, the current design would need:

ITEM	TODAY	WHY
Folder listings	40+	One `files.list` per folder visited, more for folders with over 1,000 items
Requests per file	2	A resumable session plus the upload, even for a 200-byte JSON file
Total requests	≈ 640	40 + 300 × 2
Running at once	1	Every request waits for the previous one
Memory per file	whole file	`readFileSync` loads the file completely before sending
App while it runs	stutters	Synchronous reads and writes run on Electron's main thread

At roughly a quarter of a second per request, that's about two and a half minutes spent waiting on round trips before counting the bytes themselves. Part 2 roughly halves the number of requests and runs several at a time.

10 Where the state lives

The main process keeps its state in 18 module-level `let` variables across five files, not counting the window and tray. Each one is visible to everything in its file, isn't tied to a particular server or session, and is never reset as a group. The renderer then keeps copies of some of them in its stores.

One fact, many homes

FACT	WHERE IT'S STORED	COPIES
Is a session running?	`currentHostingServerId` and `heartbeatInterval` (main.ts), `serverProcess` (childrenProcesses.ts), `serverRunning` and `lockStatus.status` (renderer), `status` in lock.json	6
Which files changed?	`watcherUpdates` and `localChanges` (the same Maps), `trackedFiles` (main), `trackedFiles` (renderer)	4
What the manifest says	`localManifest` in memory (one for all servers), manifest.json in the server folder; the Drive copy is never written	2
Hosting details	`publicIp` and `serverStartedAt` (serverService.ts), lock.json, `lockStatus` (renderer)	4
Ignored files	`ignoredNames` and `ignoredPatterns` (watcher-manifest.ts), `watcherBlacklist` in localStore.json, the Blacklist modal's own state	4

When one fact lives in several places, every change has to update all of them in the right order, and each bug is a place where one copy was missed: a Stop button for a server that's gone, the second stop run clearing `currentHostingServerId` mid-upload F9, the next server inheriting the previous server's manifest F14. The fix is a rule, not a library: **each fact gets one owner, and everyone else asks the owner or listens to it.**

11 Ten habits that would have prevented most of this

The 41 findings come from a handful of patterns. Each of these habits is small; together they remove most of the list.

Trace one real run by hand

Before trusting a refactor, follow one session through the code with real values, as in section 9. The empty transfer list, the missing Drive manifest and the version that never changes all show up within ten minutes.

F1 · F2 · F3 · F4

Let the compiler guard the edges

Run `tsc --noEmit` before every commit, avoid `any`, and type the IPC channels so a rename breaks the build instead of the app. `Object.entries(map)` compiled because its fallback overload returns `any`.

F1 · F20 · F31 · F36

Give every fact one owner

Decide which module owns each piece of state; everyone else asks or subscribes. Module-level variables shared between features are where state goes to drift apart.

F14 · F17 · F32

Model the session as a state machine

Name the phases (checking, downloading, running, stopping, uploading), allow each action only in the right phase, and give every action one entry point. A second Stop then does nothing instead of starting a second upload.

F5 · F9 · F21 · F30

Renew a lease for as long as you rely on it

A lock that expires must be renewed until the protected work is finished, must carry an owner ID, and must be re-checked after you claim it.

F6 · F7 · F8 · F34

Functions that read shouldn't write

`getServerLock()` repairing the lock as a side effect means that looking at a server changes it. Keep queries and commands apart.

F11

Await everything, and decide what failure means

An async call without `await` is a promise nobody is waiting for: its errors vanish and its timing is random. Every `await` needs an answer to "and if it fails?"

F12 · F21 · F27

Keep decisions pure

"Which files should move?" should be a function that takes snapshots and returns a plan, without reading globals or touching the network. Pure functions are easy to test, and tests would have caught most of the transfer bugs.

F3 · F13 · F14 · F15

Keep Electron's main thread free

The main process also drives the window. Synchronous reads and writes of multi-megabyte files freeze the app; stream them instead.

F18

Report errors one way

Pick one convention (Part 2 uses a Result type at the IPC boundary) and use it everywhere, so a caller always knows whether to check `.ok` or to catch.

F13 · F26

A Appendix: every finding

Sorted by severity. **Critical** means data loss, a corrupted world or two hosts at once in normal use, or a core feature that can't work. **High** means wrong behavior in common situations. **Medium** means edge cases or structure that actively breeds bugs. **Low** means small fixes.

Critical

F1 The transfer list is always empty

`electron/services/fileTransferService.ts:44`

`buildTransferMap()` loops over `Object.entries(filesToBeTransfered)`, but the argument is a Map. `Object.entries` only sees an object's own properties, and a Map keeps its entries elsewhere, so the loop runs zero times. (Checked in Node: `Object.entries(new Map([['a', 1]]))` is `[]`.) Nothing moves in either direction, the version is still increased and the app reports success.

LESSON TypeScript allowed it because the fallback overload of `Object.entries` accepts any object and returns `any`. Iterate a Map with `for (const [path, entry] of map)`.

F2 manifest.json never reaches Drive

`watcher-manifest.ts:10 · fileTransferService.ts:144, :286`

manifest.json is in `PROTECTED_DIRECTORIES`, so both recursive walkers skip it, and no other code uploads it. `readDriveManifest()` therefore always returns null: every download stops with "Manifest.json could not be found in the Drive folder", and every upload treats Drive as empty.

LESSON The manifest is the commit record of a sync. Uploading it last is part of the protocol, not an afterthought (Part 2, section 7).

F3 Changed files are never downloaded

`watcher-manifest.ts:133, :214`

Every entry gets `version: 0` from `manifestUpsert()`, and nothing increases it since `writeLocalManifest()` was removed. `getManifestDifference()` selects files whose version is higher, so it only finds files that are missing on one side. Edited files (region files, playerdata, level.dat) are never downloaded.

LESSON A per-file counter needs one writer that bumps it reliably. A content hash needs nobody: it changes when the content does.

F4 Manifest entries use the bare file name

`fileTransferService.ts:188–189, :332–333`

After each transfer the result is stored as `localManifest.files[fileName]`, where `fileName` is the last path segment. world/region/r.0.0.mca, world/entities/r.0.0.mca and world/poi/r.0.0.mca all write to `"r.0.0.mca"`, and the full-path entries are never updated.

LESSON Pass the full relative path down the recursion, or better, don't recurse at all: work from a flat list of paths (Part 2).

F5 The upload starts while Minecraft is still saving

`main.ts:388–390, :224–230 · childrenProcesses.ts:60–62`

`killServer()` writes `stop` to the server's input and returns immediately. stop-server, `doCleanup()` and google-logout all go straight on to `stopServer()` and the upload. Minecraft takes a few seconds to save its chunks, so region files can be read while they're half written.

LESSON "I asked it to stop" isn't "it stopped". Wait for the process's exit event before touching its files.

F6 The lock's lease runs out during transfers

`serverService.ts:170, :176–180 · main.ts:200`

"starting" lasts 60 s and isn't renewed while the download runs. "stopping" lasts 30 s, and `handleServerStop()` clears the heartbeat before calling `stopServer()`, so it's never renewed during the upload. After that, any member's app sees an expired lock, rewrites it as "offline" and can start hosting a half-transferred world.

LESSON A lease must be renewed for as long as the work it protects is running.

F7 Start only refuses "online"

`serverService.ts:24`

`startServer()` aborts only when `lockData.status === "online"`. A lock that says "starting" (someone is downloading) or "stopping" (someone is uploading) is treated as free, even while it's valid.

LESSON Write the rule positively: you may start only when the lock is "offline" or expired.

F8 A lapsed lock makes the host skip the upload

`serverService.ts:277–283`

`stopServer()` reads the lock first and returns "This server is not being hosted" when it says "offline" or "stopping". If the heartbeat failed for 30 s at any point (a Wi-Fi drop, a laptop going to sleep), another member's app has already rewritten the lock as "offline", and the host's whole session is never uploaded. The only sign is that confusing message.

LESSON The host's own session state, not a shared file anyone can overwrite, should decide whether there's something to upload.

High

F9 The stop sequence runs twice

`main.ts:347–350, :383–396, :215–248`

Pressing Stop calls `handleServerStop()`, and Java's `close` event calls it again a few seconds later. Both pass the `currentHostingServerId` check, because it's only cleared after the upload. The second run usually fails with "not being hosted", then clears `currentHostingServerId` and sends hide-progress and server-stopped while the first upload is still running; if Java exits quickly, both runs upload. The same race exists in `doCleanup()` and google-logout.

LESSON One entry point per action, guarded by the current phase, makes repeats harmless.

F10 Manual Download and Upload ignore the lock

`server-slot/index.tsx:61, :81 · main.ts:453–459`

The buttons in Drive Folders call sync-server and upload-server-folder directly. Any member can overwrite Drive while someone else is hosting, or download a world that's mid-upload.

LESSON Every path that writes shared data needs the same guard, or the guard protects nothing.

F11 Reading the lock writes it

`serverService.ts:197–257`

`getServerLock()` creates lock.json when it's missing and overwrites an expired one with "offline", using the reader's name as the host. Every app that merely displays the status can write it. When two calls overlap, the `lockInitializing` guard returns an object without `lockData`, and callers such as `startServer()` crash on `lockRes.lockData.status`.

LESSON Queries return data; commands change it. A get… function shouldn't do both.

F12 Async steps that nobody awaits

`fileTransferService.ts:221, :358`

`applyManifestChanges()` is async (it fetches the user's name) but is called without `await` after both transfers. The transfer reports success before the manifest is written, and if writing fails the error becomes an unhandled rejection.

LESSON Every promise needs an owner: await it, or hand it on deliberately to something that handles its errors.

F13 The upload ignores its own safety check

`fileTransferService.ts:351–353` · `watcher-manifest.ts:277–287, :301`

`getFilesToUpload()` returns a clear warning when Drive is newer, but `uploadServerFolder()` never checks `.success`: it passes `undefined` to `buildTransferMap()`, and the user sees "Cannot convert undefined or null to object" instead. The deleted-files list comes back as `filesremovedFiles` and is never used, so files deleted locally are never deleted on Drive.

LESSON When a function returns `{ success, … }`, the caller's first line should handle the failure branch.

F14 One in-memory manifest for every server

`watcher-manifest.ts:25–31, :41–44, :235, :268`

`localManifest` is a single module-level object. `readLocalManifest()` replaces it only when the file exists, so switching to a server without a manifest.json keeps the previous server's files and version. The checks meant for that case (`!localManifest.files`) can never be true, because `{}` is truthy.

LESSON Load the state for the thing you're working on and pass it in as a parameter, instead of keeping a global "current" copy.

F15 Downloads overwrite local edits without warning

`watcher-manifest.ts:125, :227–261`

`detectLocalChanges()` finds files changed while the app was closed, but `getLocalChanges()` is never called and `getFilesToDownload()` never looks at local changes. Files changed on both sides are overwritten, and files deleted on Drive are deleted locally even if you edited them.

LESSON Before overwriting local data, compare against the last synced state (the base), not just against the other side.

F16 Downloaded files will look modified

`fileTransferService.ts:128–129 · watcher-manifest.ts:79–82, :110`

Writing a downloaded file gives it the current time as its modified time, while the manifest keeps the uploader's time, so `detectLocalChanges()` flags every downloaded file as changed. If the watcher is running, its events for downloaded files arrive 3 s late, after `applyManifestChanges()` cleared the list, and land in it again.

LESSON After a sync, record what is actually on your disk (size, modified time, hash), not what the other machine had.

F17 Starting the watcher floods the app

`watcher-manifest.ts:76, :130 · store.ts:190–194 · manifest-modal/index.tsx:72`

With `ignoreInitial: false`, a 10,000-file server produces 10,000 add events, 10,000 IPC messages and 10,000 renderer updates, each copying the whole Set (about 50 million element copies). An open Manifest modal re-fetches and re-stats the list after every event.

LESSON Batch events (collect for 300–500 ms, send one message) and never copy a growing collection inside a loop.

F18 Big files are read and written synchronously on the main thread

`fileTransferService.ts:128–129, :257`

`downloadFile()` buffers the whole response and calls `writeFileSync()`; `uploadFile()` calls `readFileSync()` on the whole file. Region files are often several megabytes. Electron's main process also drives the window, so these calls freeze the app, and every file is held fully in memory.

LESSON Stream files (`createReadStream`, `pipeline`) and use the async fs API in the main process.

F19 Transfers are sequential and use extra requests

`fileTransferService.ts:233–274, :74–95, :137, :279`

Every file waits for the previous one. Every upload opens a resumable session (2 requests) even for tiny files, and every folder is listed again on every transfer. For a few hundred files that's minutes of waiting on round trips.

LESSON Store Drive file IDs in the manifest so files can be addressed directly, use one-request uploads for small files, and run several transfers at once.

F20 The project doesn't build

`main.ts:27 · manifest-modal/index.tsx:82, :122 · package.json`

`tsc --noEmit` reports three errors: main.ts imports `writeLocalManifest`, which was renamed, and the Manifest modal reads `toAdd` / `toRemove`, which are now `changedFiles` / `removedFiles`. chokidar 5 is installed but missing from package.json and package-lock.json (npm marks it "extraneous"), so a fresh clone won't have it. It also declares Node 20.19 or newer, while Electron 30.5.1 ships Node 20.16.

LESSON Add `"typecheck": "tsc --noEmit"` to package.json and run it before every commit; install packages with `npm install <name>` so they're recorded.

Medium

F21 Waiting for playit.gg can hang or escape

`serverService.ts:48–56` · `childrenProcesses.ts:70–83` · `app-cockpit/index.tsx:78–79`

`Promise.all` waits for both the download and the playit.gg address. `waitForPlayitLink()` has no timeout, so an agent that never prints an address hangs the start forever. If it rejects, the exception leaves `startServer()` unhandled: the lock stays "starting", the download may keep running, and the Power button keeps spinning because `handleStart()` has no try/catch.

LESSON Give every wait a timeout, and put cleanup in `finally`.

F22 Built-in files aren't ignored on a fresh install

`watcher-manifest.ts:72, :155`

`startWatcher()` builds `ignoredNames` from the saved blacklist only, without adding `PROTECTED_DIRECTORIES`. On a fresh install manifest.json is tracked as a change until the Blacklist tab has been opened once, and manifest.tmp is never ignored.

LESSON Built-in rules live in code and are added to the user's rules, never replaced by them.

F23 Blacklisting a nested file with Browse never matches

`blacklist-modal/index.tsx:49` · `watcher-manifest.ts:50–63`

Browse produces a Windows path such as `world\data\DistantHorizons.sqlite`. `makeSpecificDirRule()` escapes the backslashes into the regex, but `isIgnored()` tests paths that use forward slashes, so the rule never matches. Only top-level names, or paths typed with `/`, work.

LESSON Normalize every path to one format (forward slashes, relative to the server folder) the moment it enters the program.

F24 The blacklist is global and private

`localServerStore.ts:18` · `watcher-manifest.ts:164–170`

One `watcherBlacklist` is stored in this PC's localStore.json and applied to every server. Other members have their own lists, so one member skips DistantHorizons.sqlite and the next host uploads it.

LESSON Rules that decide what's shared must be shared themselves: store them with the server on Drive.

F25 Manifest paths use the OS separator

`watcher-manifest.ts:128`

`path.relative()` returns `world\region\r.0.0.mca` on Windows. A member on macOS or Linux would produce `world/region/r.0.0.mca`, and every entry would look new to the other side.

LESSON Same as F23: one canonical path format in all stored data.

F26 Errors are reported three different ways

`googleAuthService.ts:114–117` · `googleDriveService.ts:233–234, :317–318` · `server-slot/index.tsx:41–43`

Some functions throw (`authorizedFetch`, `inviteUserToServer`, `renameServerFolder`), some return `{ success: false, error }`, and the folder walkers log with `console.warn` and carry on. IPC turns a thrown error into a rejected `invoke()`, which most components don't catch; ServerSlot's rename checks `res.success` on a function that throws instead.

LESSON Choose one convention. Part 2 throws inside the main process and converts to a Result at the IPC boundary, in exactly one place.

F27 The heartbeat can overlap and fail silently

`main.ts:367–372`

`setInterval(async …)` starts a heartbeat every 20 s whether or not the previous one finished. Each run makes three requests; on a slow network they overlap, and an error becomes an unhandled rejection with no retry and no warning to the host.

LESSON Schedule the next run when the previous one ends (a `setTimeout` chain), and treat a failed renewal as an event the session must react to.

F28 A new OAuth client for every call

`googleAuthService.ts:18–46`

`getOAuthClient()` reads client_secret.json and token.json from disk and builds a new client each time; each client refreshes the token on its own and rewrites token.json. With parallel transfers, several refreshes would run at once and race on that file.

LESSON Create one authenticated client per app run and share it; refresh behind a single in-flight promise.

F29 Two services import each other

`fileTransferService.ts:8 · watcher-manifest.ts:7, :25`

fileTransferService imports from watcher-manifest, which imports `readDriveManifest` back. It works because of how ES modules bind names, but it's fragile, and fileTransferService changes watcher-manifest's exported `localManifest` directly (`.files[…]` , `.version`).

LESSON Dependencies should point one way. If two modules need each other, the shared part belongs in a third module.

F30 main.ts does everything

`main.ts:188–248, :262–298`

547 lines combine window and tray setup, the quit flow, 43 IPC handlers, session orchestration, the heartbeat and player-list parsing. The shutdown sequence (kill playit.gg, stop the server, upload, clear the heartbeat) is written three times, in `handleServerStop()` , `doCleanup()` and google-logout, and the copies have already drifted apart.

LESSON When you copy a sequence a second time, make it a function; a third time, give it a module that owns it.

F31 IPC channels are plain strings

`preload.ts:4–23 · watcherListener.ts:5`

43 request channels and 12 events are typed as strings on both sides, with untyped payloads. That's how the `toAdd` → `changedFiles` rename reached the UI unnoticed, how the `opertion` typo became part of the protocol, and how three dead connections went unnoticed.

LESSON Declare channels and payload types once, in a file both processes import.

F32 The same fact is stored in several places

`see section 10`

Whether a session is running is stored in six places, the changed-file list in four, the ignore rules in four. Each copy is updated by different code at different times.

LESSON One owner per fact; others ask the owner or subscribe to it.

F33 The cockpit's ownership check is always true

`app-cockpit/index.tsx:20, :195`

`isOwner = lockStatus.hostEmail === userEmail || !loadingServers` is true for everyone once servers have loaded, and `<DirectoryMenu isOwner/>` passes the literal `true` rather than the variable. Members see a Stop button for someone else's session; the main process refuses it, but the UI shouldn't offer it.

LESSON In JSX, a bare `isOwner` means `isOwner={true}`; write `isOwner={isOwner}`.

F34 Lock expiry depends on every PC's clock

`serverService.ts:170, :235–238`

The writer computes `expiresAt` from its own clock and every reader compares it with theirs. A PC whose clock is a minute off sees live locks as expired, or dead ones as live.

LESSON Judge time with one clock. Drive's `modifiedTime` and the `Date` header of the same response both come from Google.

F35 One vanished file fails the whole step

`watcher-manifest.ts:191–197`

`formatWatcherUpdates()` calls `fs.statSync()` on every changed path without a try/catch. If any file disappeared after its event (temporary files, logs the server rotates), the call throws: the upload stops, or the Manifest modal's request fails.

LESSON Files can change between any two lines of your code. Handle "file not found" wherever you touch the disk.

Low

F36 Typos that became part of the API

`watcherListener.ts:5 · watcher-manifest.ts:130 · googleAuthService.ts:188 · store.ts:53`

`opertion` (the event payload), `filesremovedFiles`, `succesfulDownloads`, `WatcherManifet`. One changes behavior: `logoutGoogle()` checks `tokens.acces_token`, which never exists, so the token is never revoked at Google on logout.

LESSON A spell-checker extension for your editor catches identifier typos; types catch the rest.

F37 Effects that leak

`manifest-modal/index.tsx:58–70 · greeting-section/index.tsx:28–35 · main-page/index.tsx:54–58 · server-console/index.tsx:57–58`

The Manifest modal's interval is created inside a `setTimeout` callback, so the cleanup it returns is ignored and the interval runs forever. Several `useEffect` calls add IPC listeners and never remove them.

LESSON Every effect that subscribes returns a function that unsubscribes.

F38 A failed start leaves the progress panel open

`main.ts:314–316`

start-server returns early on failure without sending hide-progress.

LESSON Put UI cleanup in `finally`.

F39 eula.txt is rewritten on every launch

`childrenProcesses.ts:44`

`launchServer()` writes eula.txt every time, so it always shows up as a changed file.

LESSON A write that changes nothing still counts as a change to a watcher. Write only when the content differs.

F40 Leftovers and repeated work

`README.md` · `src/pages/default-page.tsx` · `progress-modal/index.tsx:40–44` · `googleDriveService.ts:155–168`

The README is still the Vite template, default-page.tsx is unused, ProgressModal renders a Map directly (React would throw), and `getJoinedServers()` fetches each folder's permissions twice and calls the userinfo endpoint once per server.

LESSON Delete scaffolding as soon as it stops being useful; it costs attention every time you read the code.

F41 The playit.gg check can block a start

`serverService.ts:38`

`if (playitggPath && !playitggProcess)` fails the start when a path is saved but playit.gg is turned off. The Scripts screen clears the path when you turn it off, which is why it rarely shows.

LESSON Check the same condition you used to decide whether to launch.

Continue with Part 2, Drive Launcher Blueprint: the same app with a structure that removes most of these findings at the root, explained step by step.